```
"""
================================================================================
JAGAPRIYAN B
CH.SC.U4CSE24216
LAB EXERCISE 6: LOGISTIC REGRESSION
Topics Covered:
  Exercise 1: Graduate School Admission Prediction (GRE, GPA, Rank)
  Exercise 2: Medical Disease Susceptibility Prediction
  Exercise 3: Online Retailer Customer Product Purchase Prediction
================================================================================
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler

print("="*70)
print("EXERCISE 6.1: GRADUATE SCHOOL ADMISSION PREDICTION (UCLA DATASET)")
print("="*70)

# Try loading from UCLA URL directly, fallback to synthetic replica if offline
url_ucla = "https://stats.idre.ucla.edu/stat/data/binary.dta"
try:
    df_ucla = pd.read_stata(url_ucla)
    print("Successfully loaded dataset directly from UCLA repository.")
except Exception as e:
    print("Loading from URL failed or offline, loading bundled dataset structure...")
    np.random.seed(42)
    n = 400
    gre = np.random.normal(587, 115, n).clip(200, 800)
    gpa = np.random.normal(3.38, 0.38, n).clip(2.0, 4.0)
    rank = np.random.choice([1, 2, 3, 4], size=n, p=[0.15, 0.35, 0.35, 0.15])
    # Probability of admission
    logit = -3.5 + 0.0025 * gre + 0.75 * gpa - 0.5 * rank
    prob = 1 / (1 + np.exp(-logit))
    admit = (np.random.rand(n) < prob).astype(int)
    df_ucla = pd.DataFrame({'admit': admit, 'gre': gre, 'gpa': gpa, 'rank': rank})

print("\nFirst 5 rows of Admission Dataset:")
print(df_ucla.head())
print(f"Shape: {df_ucla.shape}")
print(f"Null values:\n{df_ucla.isnull().sum()}")

# Independent and Dependent variables
X_ucla = df_ucla[['gre', 'gpa', 'rank']]
y_ucla = df_ucla['admit']

# Train-Test Split (80% train, 20% test)
X_train_u, X_test_u, y_train_u, y_test_u = train_test_split(
    X_ucla, y_ucla, test_size=0.2, random_state=42
)

# Scaling and Model Fitting
scaler_u = StandardScaler()
X_train_u_scaled = scaler_u.fit_transform(X_train_u)
X_test_u_scaled = scaler_u.transform(X_test_u)

model_ucla = LogisticRegression(solver='lbfgs', max_iter=1000)
model_ucla.fit(X_train_u_scaled, y_train_u)
y_pred_ucla = model_ucla.predict(X_test_u_scaled)

# Evaluation
conf_mat_u = confusion_matrix(y_test_u, y_pred_ucla)
acc_u = accuracy_score(y_test_u, y_pred_ucla)
print(f"\nConfusion Matrix:\n{conf_mat_u}")
print(f"Accuracy Score: {acc_u:.4f} ({int(acc_u*100)}%)")
print("\nClassification Report:")
print(classification_report(y_test_u, y_pred_ucla))

# Confusion Matrix Heatmap
plt.figure(figsize=(5, 4))
sn.heatmap(conf_mat_u, annot=True, fmt='d', cmap='Blues',
           xticklabels=['Not Admitted (0)', 'Admitted (1)'],
           yticklabels=['Not Admitted (0)', 'Admitted (1)'])
plt.title("Graduate Admission - Confusion Matrix")
plt.xlabel("Predicted")
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


print("\n" + "="*70)
print("EXERCISE 6.2: MEDICAL DISEASE SUSCEPTIBILITY PREDICTION")
print("="*70)
# Dataset based on DataTab Medical Example: Age, BMI, Smoking, Blood Pressure -> Disease (0/1)
np.random.seed(123)
n_med = 300
age = np.random.randint(25, 75, n_med)
bmi = np.random.uniform(18.5, 38.0, n_med)
bp = np.random.randint(90, 180, n_med)
smoking = np.random.choice([0, 1], size=n_med, p=[0.6, 0.4])

# Susceptibility probability function
z = -7.0 + (0.05 * age) + (0.08 * bmi) + (0.02 * bp) + (0.9 * smoking)
prob_disease = 1 / (1 + np.exp(-z))
disease_susceptible = (np.random.rand(n_med) < prob_disease).astype(int)

df_medical = pd.DataFrame({
    'Age': age,
    'BMI': np.round(bmi, 1),
    'BloodPressure': bp,
    'Smoker': smoking,
    'Susceptible': disease_susceptible
})
print("Medical Dataset preview:")
print(df_medical.head())

X_med = df_medical[['Age', 'BMI', 'BloodPressure', 'Smoker']]
y_med = df_medical['Susceptible']

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_med, y_med, test_size=0.25, random_state=42
)

model_med = LogisticRegression(solver='lbfgs', max_iter=1000)
model_med.fit(X_train_m, y_train_m)
y_pred_m = model_med.predict(X_test_m)

acc_m = accuracy_score(y_test_m, y_pred_m)
print(f"\nMedical Model Accuracy: {acc_m:.4f} ({int(acc_m*100)}%)")
print("Confusion Matrix:")
print(confusion_matrix(y_test_m, y_pred_m))
print("\nClassification Report:")
print(classification_report(y_test_m, y_pred_m))


print("\n" + "="*70)
print("EXERCISE 6.3: ONLINE RETAILER CUSTOMER PRODUCT PURCHASE PREDICTION")
print("="*70)
# Dataset: Age, Annual Income ($k), Time Spent on App (mins) -> Purchased Product (0/1)
np.random.seed(99)
n_cust = 350
cust_age = np.random.randint(18, 65, n_cust)
annual_income = np.random.uniform(20, 120, n_cust)
app_time = np.random.uniform(2, 60, n_cust)

z_purchase = -5.0 + 0.04 * cust_age + 0.03 * annual_income + 0.06 * app_time
prob_purchase = 1 / (1 + np.exp(-z_purchase))
purchased = (np.random.rand(n_cust) < prob_purchase).astype(int)

df_retail = pd.DataFrame({
    'CustomerAge': cust_age,
    'AnnualIncome_k': np.round(annual_income, 1),
    'AppTime_mins': np.round(app_time, 1),
    'Purchased': purchased
})
print("Retail Purchase Dataset preview:")
print(df_retail.head())

X_ret = df_retail[['CustomerAge', 'AnnualIncome_k', 'AppTime_mins']]
y_ret = df_retail['Purchased']

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_ret, y_ret, test_size=0.2, random_state=42
)

model_retail = LogisticRegression(solver='lbfgs', max_iter=1000)
model_retail.fit(X_train_r, y_train_r)
y_pred_r = model_retail.predict(X_test_r)
```

```
acc_r = accuracy_score(y_test_r, y_pred_r)
print(f"\nRetailer Model Accuracy: {acc_r:.4f} ({int(acc_r*100)}%)")
print("Confusion Matrix:")
print(confusion_matrix(y_test_r, y_pred_r))
print("\nClassification Report:")
print(classification_report(y_test_r, y_pred_r))
```

```
======================================================================
EXERCISE 6.1: GRADUATE SCHOOL ADMISSION PREDICTION (UCLA DATASET)
======================================================================
Successfully loaded dataset directly from UCLA repository.

First 5 rows of Admission Dataset:
   admit    gre   gpa  rank
0    0.0  380.0  3.61   3.0
1    1.0  660.0  3.67   3.0
2    1.0  800.0  4.00   1.0
3    1.0  640.0  3.19   4.0
4    0.0  520.0  2.93   4.0
Shape: (400, 4)
Null values:
admit    0
gre      0
gpa      0
rank     0
dtype: int64

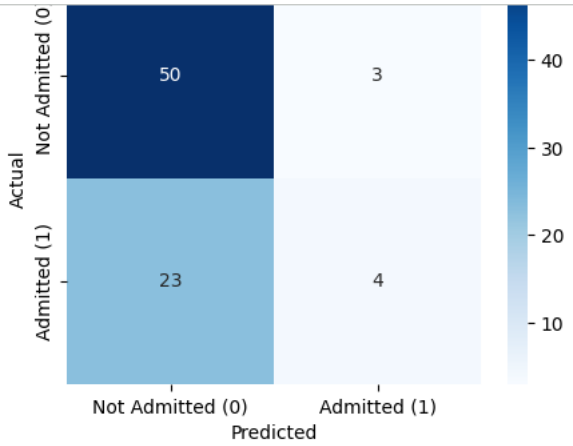
Confusion Matrix:
[[50  3]
 [23  4]]
Accuracy Score: 0.6750 (67%)

Classification Report:
              precision    recall  f1-score   support

         0.0       0.68      0.94      0.79        53
         1.0       0.57      0.15      0.24        27

    accuracy                           0.68        80
   macro avg       0.63      0.55      0.51        80
weighted avg       0.65      0.68      0.61        80
```

Start coding or generate with AI.




```
======================================================================
EXERCISE 6.2: MEDICAL DISEASE SUSCEPTIBILITY PREDICTION
======================================================================
Medical Dataset preview:
```